



香港中文大學 (深圳)
The Chinese University of Hong Kong

CSC 5003 Algorithm Art and Programming Practice

Lecture 12: Two-pointer & Sweepline

Jingbang Chen
School of Data Science (SDS)
The Chinese University of Hong Kong, Shenzhen



Minimum Window Substring

- ▶ Given two strings s and t , return the minimum window substring of s such that every character in t (including duplicates) is included in the window.
- ▶ If there is no such substring, return the empty string "".
- ▶ If there are multiple valid minimum windows, returning any one of them is acceptable.



Solution

Two-pointer or Sliding Window

- ▶ So we maintain a window: $s[\text{left}..\text{right}]$ and expand the right pointer until the window becomes valid.
- ▶ Once it is valid, we try to shrink the left pointer as much as possible while keeping it valid.
- ▶ This is the standard “expand to satisfy, shrink to optimize” sliding window pattern.



Solution

- ▶ The right pointer only moves forward, so every possible candidate window is considered when it first becomes valid.
- ▶ Once a window is valid, shrinking from the left greedily is correct because removing unnecessary prefix characters can only improve the answer.
- ▶ For a fixed right endpoint, this process finds the shortest valid window ending there. Taking the best among all such windows gives the global minimum.



Substring with Concatenation of All Words

- ▶ You are given a string s and an array of strings $words$.
- ▶ All strings in $words$ have the same length.
- ▶ A substring of s is called valid if it is formed by concatenating **all** the strings in $words$ exactly once, in **any order**, with no extra characters in between.
- ▶ Return all starting indices of valid substrings in s .
- ▶ You may return the answer in any order.



Solution

- ▶ This is a **sliding window** problem, but not at the character level.
- ▶ Since every word has the same length, a valid substring must have total length:
 - $\text{totalLen} = \text{wordLen} * \text{words.size}()$
- ▶ Also, we should not slide one character at a time in the usual way. Instead, we should process the string in **word-sized blocks**.
- ▶ Then every valid concatenation must start at an index with one of these offsets, so we run a sliding window separately for each starting offset.



Sliding Window Maximum

- ▶ You are given an integer array `nums`, and there is a sliding window of size `k` that moves from the left of the array to the right.
- ▶ You can only see the `k` numbers inside the window at each step.
Each time the window moves right by one position.
- ▶ Return an array containing the maximum value in each sliding window.



Solution

- ▶ The standard solution uses a **monotonic deque**.
- ▶ The challenge is:
 - when the window moves, how can we get the maximum of the current window efficiently?
- ▶ If we recompute the maximum from scratch for every window, the complexity becomes too large.
- ▶ So we need a data structure that supports:
 - inserting a new element on the right,
 - removing expired elements on the left,
 - retrieving the maximum quickly.
- ▶ A monotonic deque does exactly that.



Solution

- ▶ The deque stores **indices**, not values.
- ▶ We maintain it so that the corresponding values are in **decreasing order**:
 - $\text{nums}[\text{dq}[0]] \geq \text{nums}[\text{dq}[1]] \geq \text{nums}[\text{dq}[2]] \geq \dots$
- ▶ This means: the front of the deque is always the index of the maximum element in the current window.
- ▶ Now the remaining part is to do a standard two-pointer scanning.



The Skyline Problem

- ▶ A city's skyline is formed by a collection of rectangular buildings.
- ▶ You are given an array `buildings`, where each building is represented as: `[left, right, height]` meaning:
 - the building starts at x-coordinate `left`, ends at x-coordinate `right`, and has height `height`.
- ▶ Each building covers the half-open interval: `[left, right)`
- ▶ Your task is to return the **key points** of the skyline formed by these buildings.
- ▶ A key point is a point: `[x, y]` such that the height of the skyline changes to `y` at position `x`. The returned skyline should:
 - be sorted by x-coordinate,
 - contain no consecutive points with the same height.
- ▶ The last point of the skyline should always have height 0, marking the end of the skyline.



Solution

- ▶ This is a classic **sweep line** problem.
- ▶ We scan the x-axis from left to right and maintain the set of buildings that are currently active.
- ▶ At any x-coordinate, the skyline height is simply:
 - the maximum height among all active buildings.
- ▶ So the only times the skyline changes are when:
 - a building starts,
 - a building ends.
- ▶ These are the **events** we need to process.



Solution

- ▶ Each building $[l, r, h]$ generates two events:
 - a **start event** at $x = l$ with height h
 - an **end event** at $x = r$ with height h
- ▶ As we sweep from left to right:
 - when we encounter a start event, that height becomes active
 - when we encounter an end event, that height is removed from the active set
- ▶ At each step, the current skyline height is the maximum active height.



Solution

- ▶ We need a structure that supports:
 - inserting a height,
 - removing a height,
 - getting the maximum height.
- ▶ A natural choice is:
 - a max heap, together with lazy deletion, or
 - a multiset / balanced BST.
- ▶ A very clean implementation uses a **multiset**:
 - insert heights when buildings start,
 - erase one copy when buildings end,
 - the current maximum is the largest value in the multiset.
- ▶ We also keep height 0 in the multiset initially, representing the ground.



Solution

- ▶ If multiple events happen at the same x-coordinate, the order matters.
- ▶ To handle this cleanly, we can represent:
 - start events as negative heights: $(x, -h)$
 - end events as positive heights: $(x, +h)$
- ▶ Then sort all events by:
 - x-coordinate ascending
 - height ascending
- ▶ This gives the correct behavior automatically:
 - starts come before ends at the same x, because negative heights come first
 - among starts, taller buildings come first
 - among ends, shorter buildings come first
- ▶ This avoids producing incorrect intermediate key points.



Solution

- ▶ Let `prevMax` be the previous skyline height.
- ▶ For each event (x, h) :
 - if $h < 0$, it is a start event, so insert $-h$
 - otherwise it is an end event, so remove one copy of h
- ▶ Then compute:
 - `curMax` = current maximum active height
- ▶ If `curMax` $\neq$ `prevMax`, the skyline changed, so append: $[x, \text{curMax}]$ and update `prevMax` = `curMax`.



Covered Points Count

- ▶ You are given n segments on the number line. The i -th segment has integer endpoints: $[l_i, r_i]$
- ▶ A segment may degenerate into a single point, and segments may overlap, nest, or coincide. A segment $[l_i, r_i]$ covers an integer point x if and only if:
 - $l_i \leq x \leq r_i$
- ▶ For every integer k from 1 to n , compute the number of integer points that are covered by **exactly k segments**.



Solution

- ▶ This is a classic **sweep line on integer coordinates**.
- ▶ The important observation is:
 - the coverage count only changes at segment boundaries.
- ▶ Because the segments are **closed** and we are counting **integer points**, for a segment $[l, r]$ we should add:
 - +1 at l
 - -1 at $r + 1$
- ▶ Why $r + 1$?
- ▶ Because every integer point from l through r must still be covered, and the coverage should drop only after r .
- ▶ So we turn all segments into events on the number line.



Solution

- ▶ For each segment $[l, r]$, add:
 - $\text{events}[l] += 1$
 - $\text{events}[r + 1] -= 1$
- ▶ After sorting all event positions, we sweep from left to right.
- ▶ Suppose the current event position is x , and the next event position is y .
- ▶ Then for every integer point in:
 - $x, x+1, \dots, y-1$
- ▶ the coverage count is constant.
- ▶ So if the current active coverage is cur , then this block contributes $y - x$ integer points to $\text{answer}[\text{cur}]$.



Solution

Sweep Procedure

1. Build all events:
 - `+1` at each `l`
 - `-1` at each `r+1`
2. Sort event positions
3. Maintain:
 - `cur` = current coverage count
4. For each consecutive pair of event positions `x < y`:
 - first apply the change at `x`
 - then coverage on `[x, y-1]` is `cur`
 - so add `y - x` to `ans[cur]`
5. Output `ans[1..n]`

A convenient implementation is to first accumulate all changes at the same coordinate using a map.



Vika and Segments

- ▶ Vika has an infinite sheet of squared paper. Initially, all cells are white.
- ▶ She draws n black segments, each of which is either horizontal or vertical and parallel to the coordinate axes. Every segment has width equal to exactly one cell, so each segment paints a set of neighboring grid cells lying in one row or one column. Segments may overlap, touch, or even coincide.
- ▶ Each segment is given by the coordinates of its endpoints:
 - $x_1\ y_1\ x_2\ y_2$
- ▶ It is guaranteed that every segment is axis-parallel.
- ▶ Your task is to compute the total number of painted cells. If a cell is painted multiple times, it should be counted only once.



Solution

- ▶ A direct union-area style simulation is impossible because the coordinates are huge. The crucial observation from the editorial is:
 - First **merge all collinear overlapping segments**:
 - merge horizontal segments on the same row,
 - merge vertical segments on the same column.
 - Then the answer becomes:
 - (sum of lengths of all merged horizontal segments)
+ (sum of lengths of all merged vertical segments)
- (number of intersections between merged horizontal and vertical segments)



Solution

- ▶ Coordinate Compression
- ▶ Point Update/Range Sum by BIT/Segment Tree